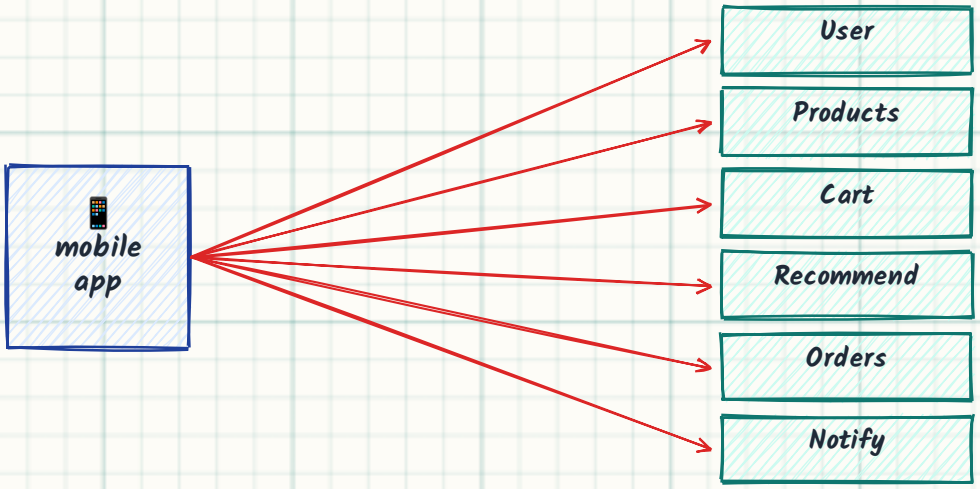
API Gateway

Lesson 8 · the front door of your microservices

In this lesson → ① Direct-call problem ② API gateway ③ Jobs of a gateway ④ Aggregation ⑤ BFF ⑥ Pitfalls ⑦ Gateway vs LB vs mesh ⑧ Spring Cloud Gateway ⑨ Interview Q&A
Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

① The problem: clients calling services directly

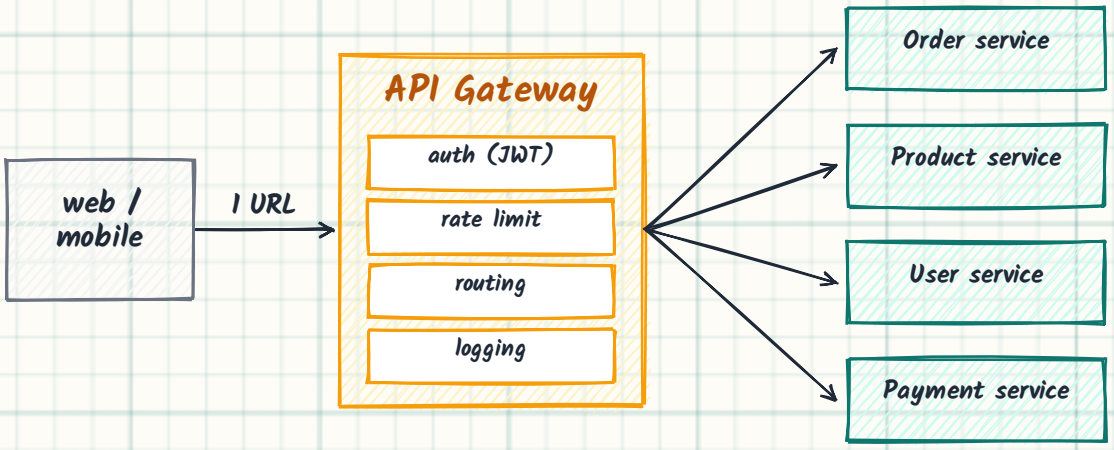


6 calls · 6 auth checks · knows every address

- ✗ **Many round trips** over slow mobile networks
- ✗ Auth, CORS, TLS **repeated** in every service
- ✗ Client **knows internal addresses** → refactoring breaks apps

② The API gateway

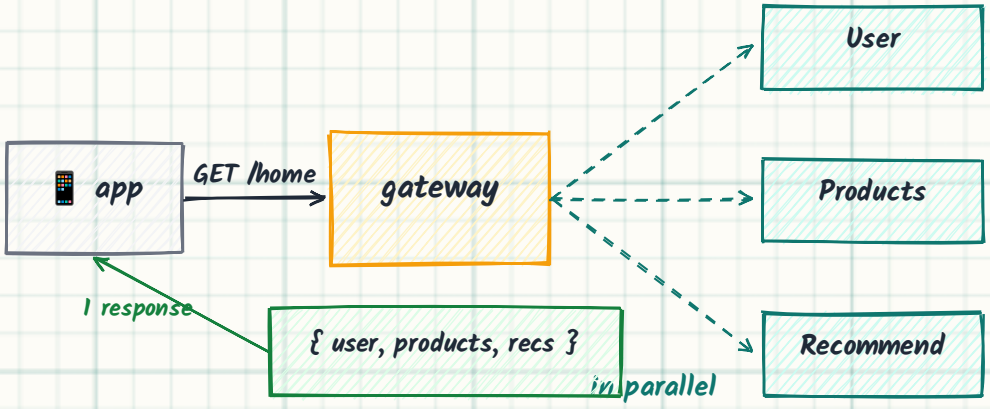
> **API gateway** = a single entry point in front of all services. Clients call one address; the gateway handles cross-cutting concerns and routes each request.



Job	What it does	ShopNest
Routing	path → service	/api/orders/** → Order service
Auth & TLS	validate token once, terminate HTTPS	check JWT, pass user id downstream
Rate limiting	protect services	100 req/min per user
Aggregation	combine several calls	GET /home = user + products + recs
Translation	REST outside → gRPC inside	Lesson 5
Observability	central logs & metrics	request IDs for tracing

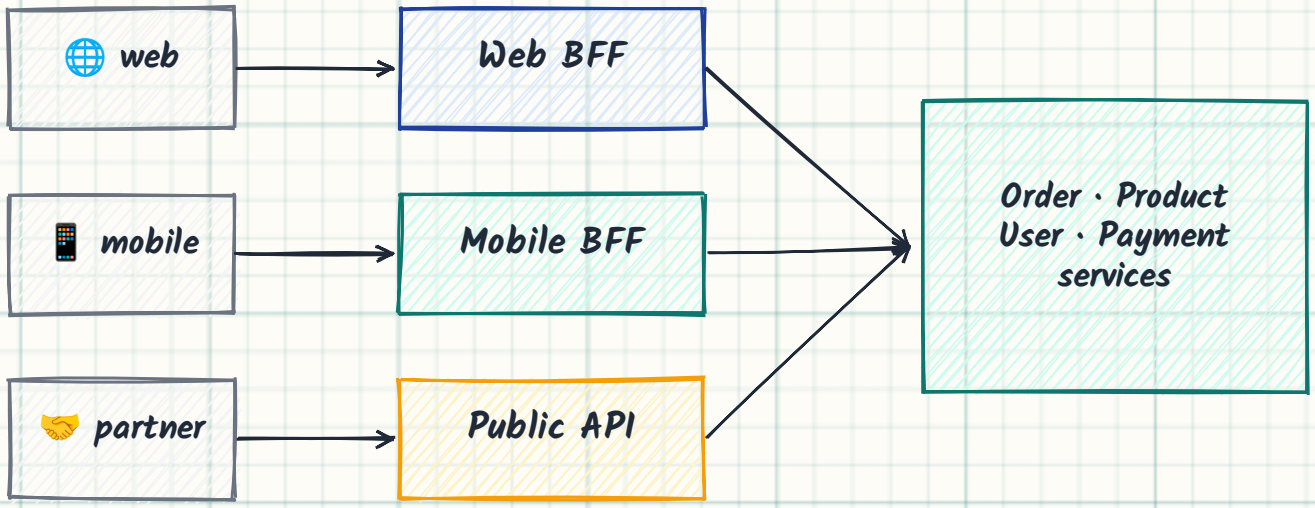
Tools: **Spring Cloud Gateway**, **Kong**, **NGINX**, **Envoy**, **AWS API Gateway**.

③ Request aggregation



④ Backend for Frontend (BFF)

> **BFF** = a separate backend *per client type* (web, mobile, partners), shaped for that client and usually owned by its frontend team.



one backend per frontend

- ✓ Mobile gets **small** payloads, web gets **rich** ones
- ✓ Frontend teams change their BFF **independently**
- ✓ Partners get a **stable** public API

When do you need a BFF?

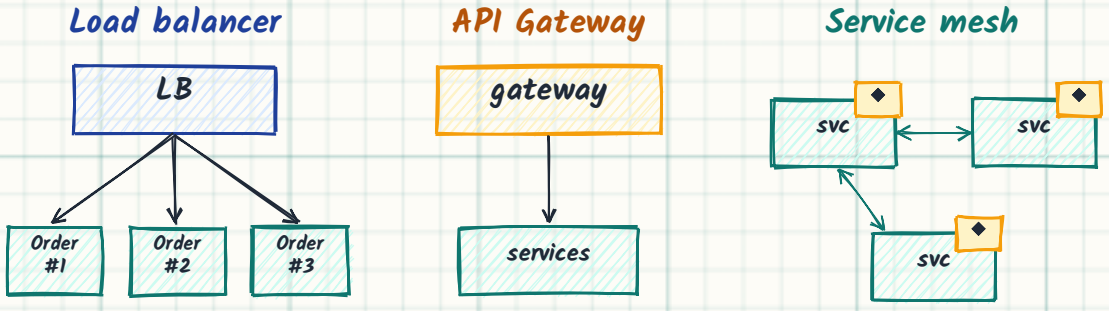
- → Clients need **very different** data or screens (web vs mobile vs TV)
- → Different **release speeds** per frontend team
- → One client type only? A single gateway is enough

Watch out
Don't copy business logic into every BFF. BFFs **shape** data; services **own** rules.

5 Pitfalls

Pitfall	Fix
Single point of failure	run multiple instances behind a load balancer
Extra latency (1 more hop)	keep it fast; cache where safe
"God gateway" with business logic	keep it thin: routing & cross-cutting only
Team bottleneck	self-service config; BFFs owned by frontend teams

6 Gateway vs load balancer vs service mesh



	Load balancer	API gateway	Service mesh
Scope	instances of 1 service	edge, into the system (north-south)	between services (east-west)
Works on	connections / requests	API paths, tokens, payloads	sidecar proxies
Examples	NGINX, cloud LB	Spring Cloud Gateway, Kong	Istio, Linkerd

⑦ Spring Cloud Gateway route

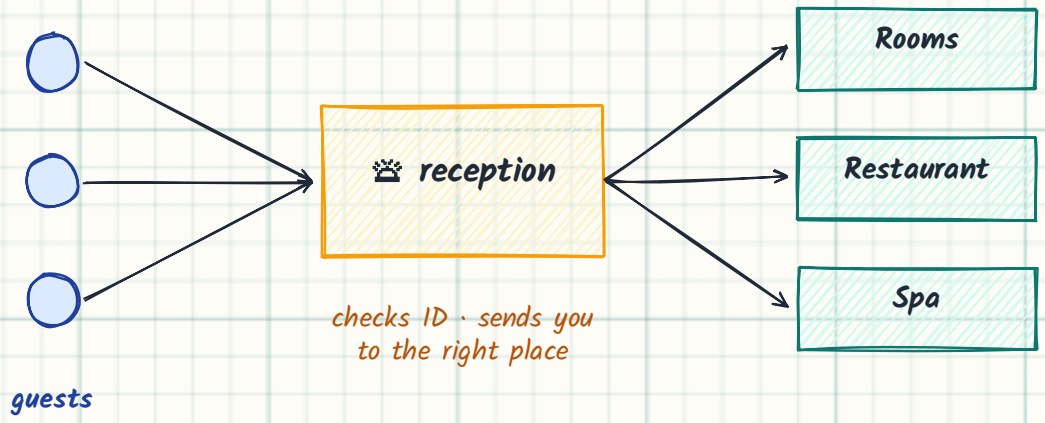
```
spring:
  cloud:
    gateway:          # (newer versions: ...gateway.server.webflux)
      routes:
        - id: orders
          uri: lb://order-service    # load-balanced via discovery
          predicates:
            - Path=/api/orders/**
          filters:
            - StripPrefix=1
            - name: RequestRateLimiter
```

→ uri: lb://order-service → *load-balanced via service discovery (Lesson 9)*

→ Path=/api/orders/** → *which requests match*

→ *Filters: strip prefix, rate limiting, add headers*

⑧ Analogy: hotel reception



Remember

Reception checks your ID and sends you to the right place. It does not cook your food.

Gateway = routing, not business logic.

Interview Corner

Q. What is an API gateway?

A. A single entry point that routes client requests to services and handles cross-cutting concerns: auth, rate limiting, logging, aggregation.

Q. What is the Backend-for-Frontend pattern?

A. A separate backend per client type (web, mobile, partner), shaped for that client and owned by its team.

Q. API gateway vs load balancer?

A. A load balancer spreads traffic across instances; a gateway works at the API level (paths, tokens, aggregation).

Q. What are the risks of an API gateway?

A. Single point of failure, extra latency, business logic creeping in, team bottleneck.

Q. Gateway vs service mesh?

A. Gateway: north-south traffic at the edge. Mesh: east-west service-to-service traffic via sidecars.

⚡ Revise in 60 seconds

- → Gateway = **single front door**
- → Routing · auth · rate limit · logs
- → **Aggregation**: 1 call instead of 6
- → **BFF**: 1 backend per client type
- → Keep it **thin** (no business logic)
- → Run **many instances**
- → LB = instances · Gateway = edge
- → Mesh = **east-west** (sidecars)

Next lesson → MS-09
Service discovery & client-side load balancing.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**